

problem and determine which kind of craftsman to dispatch. This saved the phone companies lots of money because incorrect dispatches meant delays for the customer and wasted trips for the craftsmen.

The code that made this dispatch determination was designed and written by someone who was very bright, but a terrible communicator. The process of writing the code has been described as “Three weeks of staring at the ceiling and two days of code pouring out of every orifice of his body—after which he quit.”

Nobody understood this code. Every time we tried to add a feature or fix a defect, we broke it in some way. And since it was upon this code that one of the primary economic benefits our system rested, every new defect was deeply embarrassing to the company.

In the end, our management simply told us to lock that code down and never modify it. That code became *officially rigid*.

This experience impressed upon me the value of good, clean code.

## ARCHITECTURE

The system was written in 1976 in M365 assembler. It was a single, monolithic program of roughly 60,000 lines. The operating system was a home-grown, nonpreemptive, task-switcher based on polling. We called it MPS for *multiprocessing system*. The M365 computer had no built-in stack, so task-specific variables were kept in a special area of memory and swapped out at every context switch. Shared variables were managed with locks and semaphores. Reentrancy issues and race conditions were constant problems.

There was no isolation of device control logic, or UI logic, from the business rules of the system. For example, modem control code could be found smeared throughout the bulk of the business rules and UI code. There was no attempt to gather it into a module or abstract the interface. The modems were controlled, at the bit level, by code that was scattered everywhere around the system.

The same was true for the terminal UI. Messages and formatting control code were not isolated. They ranged far and wide throughout the 60,000-line code base.

The modem modules we were using were designed to be mounted on PC boards. We bought those units from a third party, and integrated them with other circuitry onto a

board that fit into our custom backplane. These units were expensive. So, after a few years, we decided to design our own modems. We, in the software group, begged the hardware designer to use the same bit formats for controlling the new modem. We explained that the modem control code was smeared everywhere, and that our system would have to deal with both kinds of modems in the future. So, we begged and cajoled, “Please make the new modem look just like the old modem from a software control point of view.”

But when we got the new modem, the control structured was entirely different. It was not just a little different. It was entirely, and completely, different.

Thanks, hardware engineer.

What were we to do? We were not simply replacing all the old modems with new modems. Instead, we were mixing old and new modems in our systems. The software needed to be able to handle both kinds of modems at the same time. Were we doomed to surround every place in the code that manipulated the modems with flags and special cases? There were hundreds of such places!

In the end, we opted for an even worse solution.

One particular subroutine wrote data to the serial communication bus that was used to control all our devices, including our modems. We modified that subroutine to recognize the bit patterns that were specific to the old modem, and translate them into the bit patterns needed by the new modem.

This was not straightforward. Commands to the modem consisted of sequences of writes to different IO addresses on the serial bus. Our hack had to interpret these commands, in sequence, and translate them into a different sequence using different IO addresses, timings, and bit positions.

We got it to work, but it was the worst hack imaginable. It was because of this fiasco that I learned the value of isolating hardware from business rules, and of abstracting interfaces.

## **THE GRAND REDESIGN IN THE SKY**

By the time the 1980s rolled around, the idea of producing your own minicomputer and your own computer architecture was beginning to fall out of fashion. There were many microcomputers on the market, and getting them to work was cheaper and

more standard than continuing to rely on proprietary computer architectures from the late 1960s. That, plus the horrible architecture of the SAC software, induced our technical management to start a complete re-architecture of the SAC system.

The new system was to be written in C using a UNIX O/S on disk, running on an Intel 8086 microcomputer. Our hardware guys started working on the new computer hardware, and a select group of software developers, “The Tiger Team,” was commissioned with the rewrite.

I won’t bore you with the details of the initial fiasco. Suffice it to say that the first Tiger Team failed entirely after burning two or three man-years on a software project that never delivered anything.

A year or two later, probably 1982, the process was started again. The goal was the total and complete redesign of the SAC in C and UNIX on our own, newly designed, impressively powerful 80286 hardware. We called that computer “Deep Thought.”

It took years, then more years, and then even more years. I don’t know when the first UNIX-based SAC was finally deployed; I believe I had left the company by then (1988). Indeed, I’m not at all sure it ever was deployed.

Why the delay? In short, it is very difficult for a redesign team to catch up with a large staff of programmers who are actively maintaining the old system. Here’s just one example of the difficulties they encountered.

## **EUROPE**

At about the same time that the SAC was being redesigned in C, the company started to expand sales into Europe. They could not wait for the redesigned software to be finished, so of course, they deployed the old M365 systems into Europe.

The problem was that the phone systems in Europe were very different from the phone systems in the United States. The organization of the craft and of the bureaucracies were different as well. So one of our best programmers was sent to the United Kingdom to lead a team of U.K. developers to modify the SAC software to deal with all these European issues.

Of course, no serious attempt was made to integrate these changes into the U.S.-based software. This was long before networks made it feasible to transmit large code bases across the ocean. These U.K. developers simply forked the U.S.-based

code and modified it as needed.

This, of course, caused difficulties. Bugs were found on both sides of the Atlantic that needed repair on the other side. But the modules had changed significantly, so it was very difficult to determine whether the fix made in the United States would work in the United Kingdom.

After a few years of heartburn, and the installation of a high-throughput line connecting the U.S. and U.K. offices, a serious attempt was made to integrate these two forks back together again, making the differences a matter of configuration. This effort failed the first, second, and third times it was tried. The two code bases, though remarkably similar, were still too different to reintegrate—especially in the rapidly changing market environment that existed at that time.

Meanwhile, the “Tiger Team,” trying to rewrite everything in C and UNIX, realized that it also had to deal with this European/US dichotomy. And, of course, that did nothing to accelerate their progress.

## **SAC CONCLUSION**

There are many other stories I could tell you about this system, but it’s just too depressing for me to continue. Suffice it to say that many of the hard lessons of my software life were learned while immersed in the horrible assembler code of the SAC.

## **C LANGUAGE**

The 8085 computer hardware that we used in the 4-Tel Micro project gave us a relatively low-cost computing platform for many different projects that could be embedded into industrial environments. We could load it up with 32K of RAM and another 32K of ROM, and we had an extremely flexible and powerful scheme for controlling peripherals. What we did not have was a flexible and convenient language with which to program the machine. The 8085 assembler was simply not fun to write code in.

On top of that, the assembler we were using was written by our own programmers. It ran on our M365 computers, using the cartridge tape operating system described in the “Laser Trim” section.

As fate would have it, our lead *hardware* engineer convinced our CEO that we needed a *real* computer. He didn't actually know what he would do with it, but he had a lot of political clout. So we purchased a PDP-11/60.

I, a lowly software developer at the time, was ecstatic. I knew *precisely* what I wanted to do with that computer. I was determined that this was going to be *my* machine.

When the manuals arrived, many months before the delivery of the machine, I took them home and devoured them. By the time the computer was delivered, I knew how to operate both the hardware and the software at an intimate level—at least, as intimate as home study can make it.

I helped to write the purchase order. In particular, I specified the disk storage that the new computer would have. I decided we should buy two disk drives that could take removable disk packs that held 25 megabytes each.<sup>9</sup>

Fifty megabytes! The number seemed infinite! I remember walking through the halls of the office, late at night, cackling like the Wicked Witch of the West: “Fifty megabytes! Hahahahahahahahah!”

I had the facilities manager build a little room that would house six VT100 terminals. I decorated it with pictures from space. Our software developers would use this room to write and compile code.

When the machine arrived, I spent several days setting it up, wiring all the terminals, and getting everything to work. It was a joy—a labor of love.

We purchased standard assemblers for the 8085 from Boston Systems Office, and we translated the 4-Tel Micro code into that syntax. We built a cross-compilation system that allowed us to download compiled binaries from the PDP-11 to our 8085 development environments, and ROM burners. And—Bob's your Uncle—it all worked like a champ.

## C

But that left us with the problem of still using 8085 assembler. That was not a situation that I was happy with. I had heard that there was this “new” language that was heavily used at Bell Labs. They called it “C.” So I purchased a copy of *The C Programming Language* by Kernighan and Ritchie. Like the PDP-11 manuals a few

months before, I *inhaled* this book.

I was astounded by the simple elegance of this language. It sacrificed none of the power of assembly language, and provided access to that power with a much more convenient syntax. I was sold.

I purchased a C compiler from Whitesmiths, and got it running on the PDP-11. The output of the compiler was assembler syntax that was compatible with the Boston Systems Office 8085 compiler. So we had a pathway to go from C to the 8085 hardware! We were in business.

Now the only problem was convincing a group of embedded assembly language programmers that they should be using C. But that's a nightmare tale for another time ...

## BOSS

Our 8085 platform had no operating system. My experience with the MPS system of the M365, and the primitive interrupt mechanisms of the IBM System 7, convinced me that we needed a simple task switcher for the 8085. So I conceived of BOSS: Basic Operating System and Scheduler.<sup>[10](#)</sup>

The vast majority of BOSS was written in C. It provided the ability to create concurrent tasks. Those tasks were not preemptive—task switching did not take place based on interrupts. Instead, and just like with the MPS system on the M365, tasks were switched based on a simple polling mechanism. The polling happened whenever a task blocked for an event.

The BOSS call to block a task looked like this:

```
block(eventCheckFunction);
```

This call suspended the current task, placed the `eventCheckFunction` in the polling list, and associated it with the newly blocked task. It then waited in the polling loop, calling each of the functions in the polling list until one of them returned `true`. The task associated with that function was then allowed to run.

In other words, as I said before, it was a simple, nonpreemptive task switcher.

This software became the basis for a vast number of projects over the following years. But one of the first was the pCCU.

## pCCU

The late 1970s and early 1980s were a tumultuous time for telephone companies. One of the sources of that tumult was the digital revolution.

For the preceding century, the connection between the central switching office and the customer's telephone had been a pair of copper wires. These wires were bundled into cables that spread in a huge network across the countryside. They were sometimes carried on poles, and sometimes buried underground.

Copper is a precious metal, and the phone company had tons (literally tons) of it covering the country. The capital investment was enormous. Much of that capital could be reclaimed by transporting the telephone conversation over digital connections. A single pair of copper wires could carry hundreds of conversations in digital form.

In response, the phone companies embarked upon the process of replacing their old analog central switching equipment with modern digital switches.

Our 4-Tel product tested copper wires, not digital connections. There were still plenty of copper wires in a digital environment, but they were much shorter than before, and they were localized near the customer's telephones. The signal would be carried digitally from the central office to a local distribution point, where it would be converted back to an analog signal and distributed to the customer over standard copper wires. This meant that our measurement device needed to be located out where the copper wires began, but our dialing device needed to remain at the central office. The problem was that all our COLTs embodied both dialing and measurement in the same device. (We could have saved ourselves a fortune had we recognized that obvious architectural boundary a few years earlier!)

Thus we conceived of a new product architecture: the CCU/CMU (the COLT control unit and the COLT measurement unit). The CCU would be located at the central switching office, and would handle the dialing of the phone lines to be tested. The CMU would be located at the local distribution points, and would measure the copper wires that led to the customer's phone.

The problem was that for each CCU, there were many CMUs. The information about which CMU should be used for each phone number was held by the digital switch itself. Thus the CCU had to interrogate the digital switch to determine which CMU to communicate with and control.

We promised the phone companies that we would have this new architecture working in time for their transition. We knew they were months, if not years away, so we did not feel rushed. We also knew that it would take several man-years to develop this new CCU/CMU hardware and software.

## THE SCHEDULE TRAP

As time went on, we found that there were always urgent matters that required us to postpone development of the CCU/CMU architecture. We felt safe about this decision because the phone companies were consistently delaying the deployment of digital switches. As we looked at their schedules, we felt confident that we had plenty of time, so we consistently delayed our development.

Then came the day that my boss called me into his office and said: *“One of our customers is deploying a digital switch next month. We have to have a working CCU/CMU by then.”*

I was aghast! How could we possibly do man-years of development in a month? But my boss had a plan ...

We did not, in fact, need a full CCU/CMU architecture. The phone company that was deploying the digital switch was tiny. They had only one central office, and only two local distribution points. More importantly, the “local” distribution points were not particularly local. They actually had regular-old analog switches in them that switched to several hundred customers. Better yet, those switches were of a kind that could be dialed by a normal COLT. Better even still, the customer’s phone number contained all the information necessary to decide which local distribution point to use. If the phone number had a 5, 6, or 7 in a certain position, it went to distribution point 1; otherwise, it went to distribution point 2.

So, as my boss explained to me, we did not actually need a CCU/CMU. What we needed was a simple computer at the central office connected by modem lines to two standard COLTs at the distribution points. The SAC would communicate with our computer at the central office, and that computer would decode the phone number and then relay the dialing and measurement commands to the COLT at the

appropriate distribution point.

Thus was born the pCCU.

This was the first product written in C and using BOSS that was deployed to a customer. It took me about a week to develop. There is no deep architectural significance to this tale, but it makes a nice preface to the next project.

## **DLU/DRU**

In the early 1980s, one of our customers was a telephone company in Texas. They had large geographic areas to cover. In fact, the areas were so large that a single service area required several different offices from which to dispatch craftsmen. Those offices had test craftspeople who needed terminals into our SAC.

You might think that this was a simple problem to solve—but remember that this story takes place in the early 1980s. Remote terminals were not very common. To make matters worse, the hardware of the SAC presumed that all the terminals were local. Our terminals actually sat on a proprietary, high-speed, serial bus.

We had remote terminal capability, but it was based on modems, and in the early 1980s modems were generally limited to 300 bits per second. Our customers were not happy with that slow speed.

High-speed modems were available, but they were very expensive, and they needed to run on “conditioned” permanent connections. Dial-up quality was definitely not good enough.

Our customers demanded a solution. Our response was DLU/DRU.

DLU/DRU stood for “Display Local Unit” and “Display Remote Unit.” The DLU was a computer board that plugged into the SAC computer chassis and pretended to be a terminal manager board. Instead of controlling the serial bus for local terminals, however, it took the character stream and multiplexed it over a single 9600-bps conditioned modem link.

The DRU was a box placed at the customer’s remote location. It connected to the other end of the 9600-bps link, and had the hardware to control the terminals on our proprietary serial bus. It demultiplexed the characters received from the 9600-bps link and sent them to the appropriate local terminals.

Strange, isn't it? We had to engineer a solution that nowadays is so ubiquitous we never even think about it. But back then ...

We even had to invent our own communications protocol because, in those days, standard communications protocols were not open source shareware. Indeed, this was long before we had any kind of Internet connection.

## ARCHITECTURE

The architecture of this system was very simple, but there are some interesting quirks I want to highlight. First, both units used our 8085 technology, and both were written in C and used BOSS. But that's where the similarity ended.

There were two of us on the project. I was the project lead, and Mike Carew was my close associate. I took on the design and coding of the DLU; Mike did the DRU.

The architecture of the DLU was based on a dataflow model. Each task did a small and focused job, and then passed its output to the next task in line, using a queue. Think of a pipes and filters model in UNIX. The architecture was intricate. One task might feed a queue that many others would service. Other tasks would feed a queue that just one task would service.

Think of an assembly line. Each position on the assembly line has a single, simple, highly focused job to perform. Then the product moves to the next position in line. Sometimes the assembly line splits into many lines. Sometimes those lines merge back into a single line. That was the DLU.

Mike's DRU used a remarkably different scheme. He created one task per terminal, and simply did the entire job for that terminal in that task. No queues. No data flow. Just many identical large tasks, each managing its own terminal.

This is the opposite of an assembly line. In this case the analogy is many expert builders, each of whom builds an entire product.

At the time I thought my architecture was superior. Mike, of course, thought his was better. We had many entertaining discussions about this. In the end, of course, both worked quite well. And I was left with the realization that software architectures can be wildly different, yet equally effective.

# VRS

As the 1980s progressed, newer and newer technologies appeared. One of those technologies was the computer control of *voice*.

One of the features of the 4-Tel system was the ability of the craftsman to locate a fault in a cable. The procedure was as follows:

- The tester, in the central office, would use our system to determine the approximate distance, in feet, to the fault. This would be accurate to within 20% or so. The tester would dispatch a cable repair craftsman to an appropriate access point near that position.
- The cable repair craftsman, upon arrival, would call the tester and ask to begin the fault location process. The tester would invoke the fault location feature of the 4-Tel system. The system would begin measuring the electronic characteristics of that faulty line, and would print messages on the screen requesting that certain operations be performed, such as opening the cable or shorting the cable.
- The tester would tell the craftsman which operations the system wanted, and the craftsman would tell the tester when the operation was complete. The tester would then tell the system that the operation was complete, and the system would continue with the test.
- After two or three such interactions, the system would calculate a new distance to the fault. The cable craftsman would then drive to that location and begin the process again.

Imagine how much better that would be if the cable craftsmen, up on the pole or standing at a pedestal, could operate the system themselves. And that is exactly what the new voice technologies allowed us to do. The cable craftsmen could call directly into our system, direct the system with touch tones, and listen to the results being read back to them in a pleasant voice.

## THE NAME

The company held a little contest to select a name for the new system. One of the most creative of the names suggested was SAM CARP. This stood for “Still Another Manifestation of Capitalist Avarice Repressing the Proletariat.” Needless to say, that wasn’t selected.

Another was the Teradyne Interactive Test System. That one was also not selected.

Still another was Service Area Test Access Network. That, too, was not selected.

The winner, in the end, was VRS: Voice Response System.

## ARCHITECTURE

I did not work on this system, but I heard about what happened. The story I am going to relate to you is second-hand, but for the most part, I believe it to be correct.

These were the heady days of microcomputers, UNIX operating systems, C, and SQL databases. We were determined to use them all.

From the many database vendors out there, we eventually chose UNIFY. UNIFY was a database system that worked with UNIX, which was perfect for us.

UNIFY also supported a new technology called *Embedded SQL*. This technology allowed us to embed SQL commands, as strings, right into our C code. And so we did—everywhere.

I mean, it was just so cool that you could put your SQL right into your code, anywhere you wanted. And where did we want to? Everywhere! And so there was SQL smeared throughout the body of that code.

Of course, in those days SQL was hardly a solid standard. There were lots of special vendor-specific quirks. So the special SQL and special UNIFY API calls were also smeared throughout the code.

This worked great! The system was a success. The craftsmen used it, and the telephone companies loved it. Life was all smiles.

Then the UNIFY product we were using was cancelled.

Oh. Oh.

So we decided to switch to SyBase. Or was it Ingress? I don't remember. Suffice it to say, we had to search through all that C code, find all the embedded SQL and special API calls, and replace them with corresponding gestures for the new vendor.

After three months of effort or so, we gave up. We couldn't make it work. We were so coupled to UNIFY that there was no serious hope of restructuring the code at any practical expense.